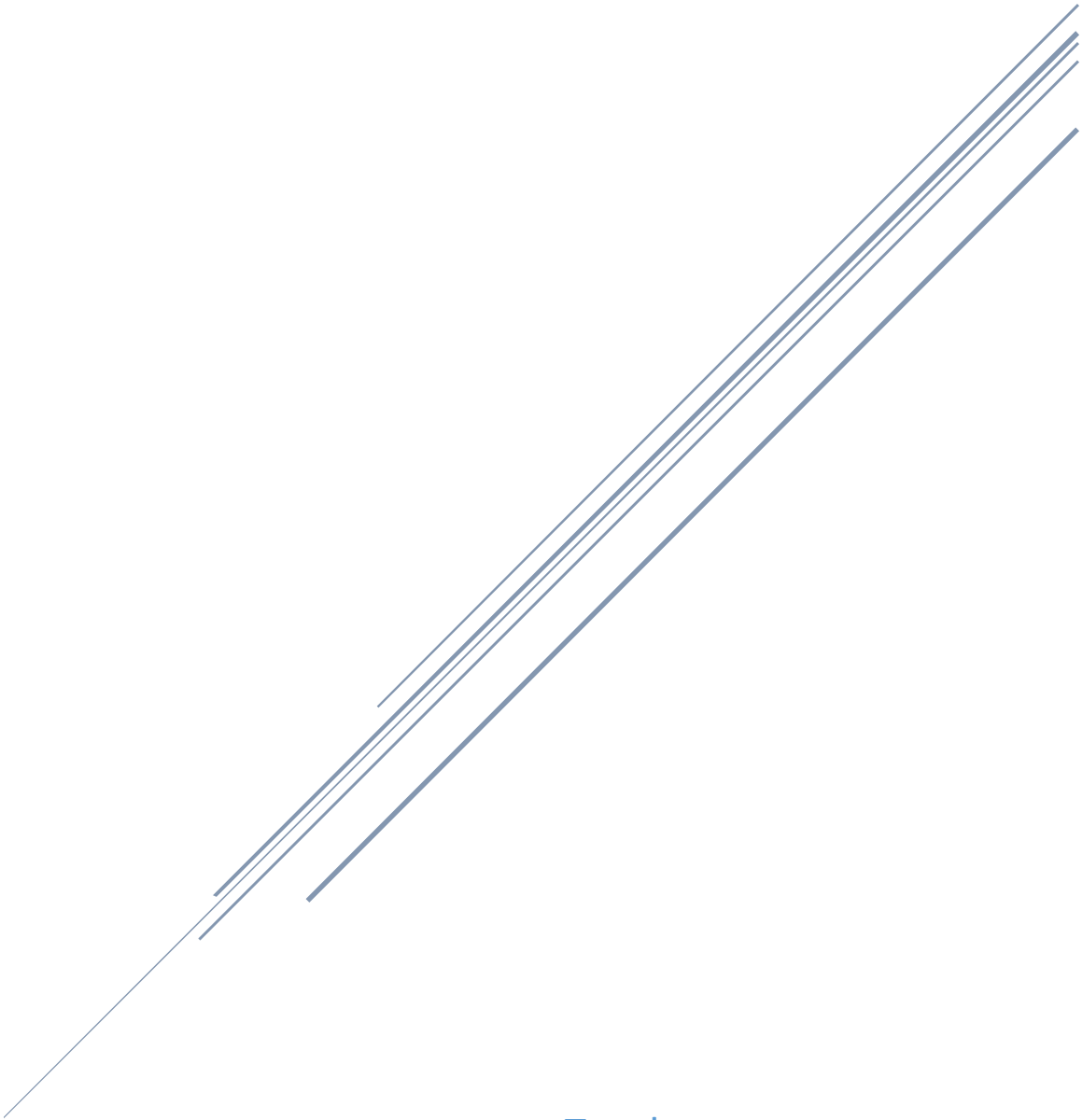
CODE AND SUMMARY OF CLASS 16

Sahir Ahmed Sheikh
Saturday (2 – 5)



Teachers:
Muhammad Bilal And Ali Aftab Sheikh

Code And Summary Of Class 16 – Saturday (2 – 5) | Quarter 3

Introduction

This document provides a detailed summary of Class 16 of Quarter 3, held on Saturday from 2 to 5 PM, led by Sir Ali Aftab Sheikh. The session began with assignment checks by the class representative (CR) and assistants. The focus was on reviewing assignment status, revising tools from the previous class, and introducing handoff concepts with practical coding demonstrations. The class revised key AI agent concepts like LLMs, agent loops, and tools, while building on prior topics such as agents and streaming. Sir Ali provided in-depth explanations of **tool types** (including **hosted tools**), **tool behavior**, **agent-as-tool** differences, and **handoff**. The session included theoretical revisions, code examples for tools and handoff, debugging tips, and motivational guidance on daily study, forming groups, and preparing for the upcoming test.

The class addressed the following key objectives:

- Reviewing and checking assignments from previous classes, noting non-submitters and warning about filtering due to incomplete work.
- Revising tools (including hosted, function, and agent-as-tool) and introducing handoff with practical examples.
- Encouraging students to experiment with code parameters, tool calls, and visualization for deeper understanding.
- Providing motivational advice on skill specialization, consistent learning, and avoiding distractions from short-term success stories.
- Highlighting the importance of practical coding, strong system prompts, source code analysis, and professional development practices for industry relevance.

Students were reminded of the upcoming test, strict assignment policy, and the need to push code to GitHub. The session stressed self-directed learning and preparation for Quarter 4's challenges.

Topics Covered

1. Class Updates and Quarter 3 Progress

- **Assignment Check and Filtering**: Reviewed three assignments from previous classes. Checked completion status, with focus on boys and girls. Implemented a filtering system to identify students who do not complete their assignments and those who are not serious.
- **Motivational Advice**: Emphasized focusing on one skill for career growth, avoiding multi-field distractions. Shared examples from Amazon VA courses and the importance of consistent income over one-time hits.
- **Test Preparation**: Upcoming test in early/mid-August with practical questions. Advised reviewing documentation, running code, and experimenting with parameters.
- **Class Merging Policy**: If assignments remain incomplete, students risk elimination or merging with another class. CRs and assistants will check assignments next week.

2. Revision Of AI Agent Concepts:

The session revised key agent concepts, emphasizing tools, hosted tools, agent-as- tool, and handoff. Key points included:

- **Tools**: Essential for tasks LLMs can't handle alone, like real-time data.Types include hosted, function, and agent tools. Difference from handoff: tools return to LLM for final output; handoff transfers full control.
- **Agent-as-Tool**: Converts agents to tools for specific tasks without transferring conversation history. Used for multi-agent systems like translators.
- **Handoff**: Transfers entire conversation and control to another agent for specialized handling.
- **LLM Concepts:** Stateless nature, conversation history, hallucination, and cutoff knowledge. Explained with examples like weather queries and web search tools.

3. AI Architecture: Tools, Hosted Tools, Agent-as-Tool, & Handoff

3.1 Tools

***Definition and Purpose**: Tools are extra functions that an AI can use to perform tasks it normally can't, like fetching live information (for example, the weather). They help the AI avoid mistakes and handle old or missing knowledge. Tools are critical because without them, an agent is just an LLM that might hallucinate or give inaccurate responses. For instance, if you ask about recent events, the LLM might make up facts, but a tool can search the web for accurate data.*

Key Features:

- **Function Calling:** Tools are defined as Python functions for specific tasks.
- **Custom vs. Hosted:** Custom (developer-defined, e.g., API calls); Hosted (pre-built, e.g., Open AI web search).
- **Integration**: Registered with agents; LLM decides invocation based on query.

How It Works:

- Four items to LLM: System prompt, user prompt, tool message, tool schema.
- Agent loop: LLM returns final output or tool call; tool response goes back to LLM (default) or directly to user.
- Max Turns: Limits loop iterations to prevent infinite loops; default 10.
- Tool Behavior: By default, tool output goes back to LLM for summarization, but can be set to stop on first tool for direct output.

Practical Application:

- In class, demonstrated an add tool to sum numbers and a weather tool to fetch data.
- **Example:**

```python
from openai_agents import FunctionTool, Agent, Runner  # Assuming library imports

@FunctionTool
def add(a: int, b: int) → int:
    print("Hello from the add tool")  # To check if called
    return a + b + 5  # Modified to test LLM override

# Create agent with tool
agent = Agent(
    name="math_agent",
    instructions="You are a helpful math assistant. Use the add tool for additions.",
    tools=[add]
)

# Run the agent
result = Runner(agent=agent).run("What is the sum of 10 and 20?")
print(result.final_output)
```

- **Explanation**: Creates a function with @FunctionTool decorator, adds parameters a and b, and a docstring like "Add two numbers". Pass to agent in tools list. When run, if tool is called, it prints the message and returns 35, but LLM might override to 30 if it disagrees. To stop LLM override, set tool_use_behavior="stop_on_first_tool" in agent settings. This shows how tools extend agent capabilities but output flows back to LLM by default for final processing. Experiment with max turns by adding agent.max_turns = 5 to see errors if exceeded.

Why Its Important:

- Enables accurate, real-time actions in AI systems.
- Prepares for Quarter 4's advanced multi-agent applications.

3.2 Hosted Tools

***Definition and Purpose**: Hosted tools are pre-built tools provided by external services or libraries, unlike custom tools you define yourself. They are "hosted" meaning they run on remote servers, handling complex tasks without you writing the code. For example, a hosted web search tool can fetch real-time internet data to answer queries beyond the LLM's training cutoff, preventing hallucinations on current events.*

Key Features:

- **Pre-Built:** No need to code the function; just integrate via API or library.
- **Examples:** OpenAI's web search, weather APIs, or database query tools.
- **Integration:** Similar to custom tools, registered with agents using their schema.

How It Works:

- LLM receives the tool schema and decides to call it like any tool.
- The hosted service processes the request (e.g., searches the web) and returns data to the LLM for summarization.
- Agent Loop: Same as regular tools; output loops back unless configured otherwise.

Practical Application:

- Demonstrated with a hypothetical hosted web search tool for recent info.
- **Example:**

```python
from openai_agents import Agent, Runner
from some_library import WebSearchTool  # Assume a hosted tool import

web_search = WebSearchTool()  # Hosted tool instance

agent = Agent(
    name="info_agent",
    instructions="You are a helpful assistant. Use web search for current events.",
    tools=[web_search]
)

result = Runner(agent=agent).run("What is the latest news on Pakistan-India relations?")
print(result.final_output)
```

- **Explanation**: The hosted WebSearchTool is pre-defined and handles API calls to search engines. When the query requires up-to-date info, LLM calls it, gets results, and summarizes. This saves development time and handles complex ops like internet access. If the LLM's knowledge is cutoff (e.g., trained up to 2023), hosted tools bridge the gap. Test by printing inside the tool if custom-wrapped, or check logs for calls.

Why Its Important:

- Provides ready-made capabilities for real-world AI, like live data integration.
- Essential for scalable systems in Quarter 4.

3.3 Agent-as-Tool

***Definition and Purpose**: Agent-as-tool converts an entire agent into a tool for specific tasks without transferring control or the full conversation history. It's used in multi-agent systems for modular functionality, like having a translator agent as a tool in a main chatbot. This keeps the main agent in control, processing the sub-agent's output like any tool response.*

Key Features:

- **No Control Transfer:** Response returns to main agent; conversation continues with original agent.
- **Difference from Handoff:** Handoff transfers history and control; agent-as-tool only processes generated input.
- **Specialization:** Ideal for tasks like translation in specific languages.

How It Works:

- Use .as_tool() on an agent to convert it.
- Main (triage) agent calls it based on instructions (e.g., "Always use provided tools").
- The sub-agent runs independently but output goes back to main LLM.

Practical Application:

- Built translator chatbot with triage agent managing language-specific tools.
- **Example:**

```
from openai_agents import Agent, Runner

# Specialist agents
spanish_translator = Agent(
    name="spanish_translator",
    instructions="Translate text to Spanish."
)

french_translator = Agent(
    name="french_translator",
    instructions="Translate text to French."
)

# Convert to tools
spanish_tool = spanish_translator.as_tool()
french_tool = french_translator.as_tool()

# Main triage agent
triage_agent = Agent(
    name="translator_agent",
    instructions="You are a translation manager. Always use the provided tools for translations. Refuse unsupported languages.",
    tools=[spanish_tool, french_tool]
)

result = Runner(agent=triage_agent).run("Translate 'Hello, how are you?' to Spanish.")
print(result.final_output)
```

- **Explanation:** Creates specialist agents with instructions, converts them using .as_tool(), and passes to main agent. For a query, triage calls the appropriate tool (e.g., spanish_tool), which runs the sub-agent on the input, returns translation, and main agent processes it. If unsupported language, refuses based on prompt. This is token-efficient for modular tasks without full handoff. Test by running different languages; add prints in sub-agents to verify calls.

Why Its Important:

- Enables modular, scalable AI systems without conflicts.
- Essential for Quarter 4's focus on collaborative agents.

3.4 Handoff

***Definition and Purpose**: Handoff transfers full control and conversation history to another agent for specialized handling. It's like delegating a task completely to an expert, saving tokens by avoiding extra LLM loops. Useful for routing queries to specialists without returning to the main agent.*

Key Features:

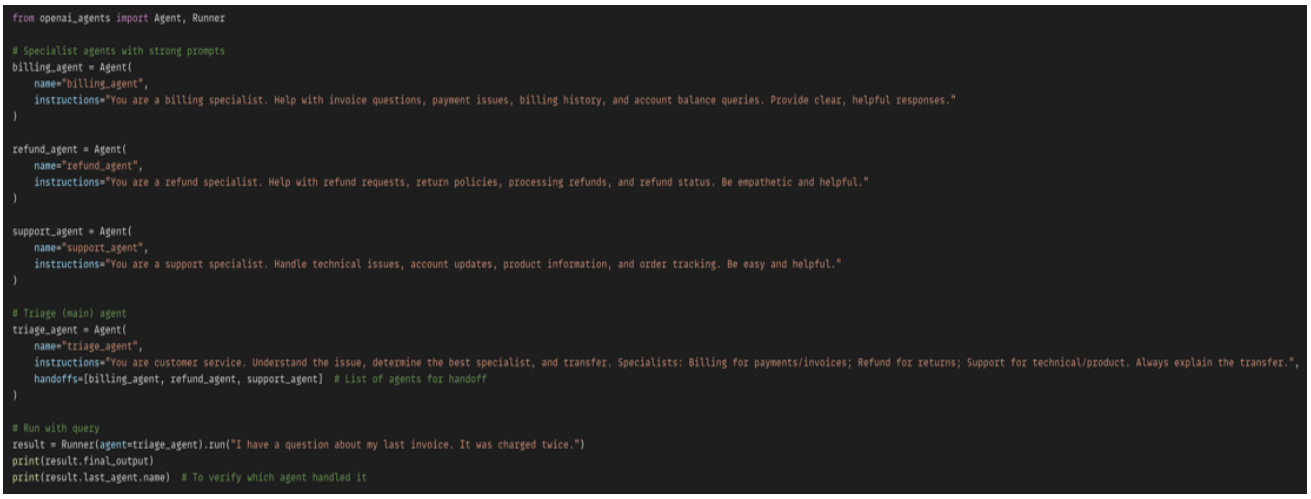
- **Full Transfer:** New agent continues conversation independently**.**
- **Use Cases:** Routing tasks to expert agents (e.g., billing queries to billing agent).

How It Works:

- Main agent calls handoff; control shifts completely.
- Difference from Tools/Agent-as-Tool: No return to main agent; new agent returns final output.

Practical Application:

- Built agents for billing, refund, support with triage managing handoffs.
- **Example:**



```python
from openai_agents import Agent, Runner

# Specialist agents with strong prompts
billing_agent = Agent(
    name="billing_agent",
    instructions="You are a billing specialist. Help with invoice questions, payment issues, billing history, and account balance queries. Provide clear, helpful responses."
)

refund_agent = Agent(
    name="refund_agent",
    instructions="You are a refund specialist. Help with refund requests, return policies, processing refunds, and refund status. Be empathetic and helpful."
)

support_agent = Agent(
    name="support_agent",
    instructions="You are a support specialist. Handle technical issues, account updates, product information, and order tracking. Be easy and helpful."
)

# Triage (main) agent
triage_agent = Agent(
    name="triage_agent",
    instructions="You are customer service. Understand the issue, determine the best specialist, and transfer. Specialists: Billing for payments/invoices; Refund for returns; Support for technical/product. Always explain the transfer.",
    handoffs=[billing_agent, refund_agent, support_agent]  # List of agents for handoff
)

# Run with query
result = Runner(agent=triage_agent).run("I have a question about my last invoice. It was charged twice.")
print(result.final_output)
print(result.last_agent.name)  # To verify which agent handled it
```

- You can zoom it and view.
- **Explanation:** Create specialists with detailed instructions defining their roles. Triage agent gets handoffs list and routes based on prompt (e.g., invoice to billing_agent). When handoff occurs, control transfers fully; output comes directly from specialist. Test by printing last_agent.name to confirm transfer (e.g., "billing_agent"). Strong prompts ensure accurate routing; weak prompts may fail. This is more efficient than agent-as-tool for ongoing conversations.

Why Its Important:

- Supports complex, multi-agent workflows for real-world applications.

---

4. Practical Coding and Debugging

- **Environment Setup**: UV project, install openai-agents, chainlit, python-dotenv. Load Gemini API key via .env.

- **Debugging**: Check required parameters (e.g., name), source code for asterisks (indicating required). Enable logging for turns/tokens; visualize agent flow with graph library.Add print statements to verify tool/handoff calls.
- **Validation Logic**: Experiment with max turns, empty strings; avoid sensitive data exposure. Handle errors like ValueError by fixing prompts or configurations.

---

5. Key Insights and Takeaways

- **AI Agent Development**: Master tools, hosted tools, agent-as-tool, handoff for practical AI.
- **Self Learning**: Experiment with code, review source code/GitHub for test prep.
- **Industry Relevance**: Focus on one skill, consistent practice for IT careers.
- **Professional Coding Practices**: Use logging/visualization for observability; push to GitHub.

---

6. Class Resources and Assignments

- **Today's Class Code**
  Students are encouraged to experiment with the provided examples to deepen understanding.
- **Complete Previous Assignments**:
  Ensure that the previous assignments are completed and pushed to GitHub, and submit the link via the assignment submission form.

Important Reminder

- Use documentation and tools like NotebookLM for test preparation, focusing on practical coding skills.
- All Agents projects must be on GitHub.
- Stay active on Discord for class announcements and review the Panaversity repository for the syllabus.

Final Words

Your responsibility is to learn actively, participate confidently, and practice consistently. Quarter 4 will demand greater effort with advanced topics like AI and cloud computing. Embrace challenges, complete assignments, and build your portfolio to seize IT industry opportunities. Stay focused, and may Allah guide you.

Allah Hafiz – See you in the Next Class!

Repo Link: GIAIC-Sat-Afternoon

This is my repository where I have uploaded all the class codes, assignments, and detailed summaries. You can go through each class code and its full detailed summary without any hesitation, and it will also help you prepare well for your test.

Thank You for Reading!

Hope you understood Class 16 well.

"True wisdom lies not in hoarding knowledge, but in igniting curiosity—to explore, to create, and to shape tomorrow." – *Sahir Ahmed Sheikh*